Prometheus 告警恢复时，怎么获取恢复时的值?

Original 巴辉特 SRETALK 2024年08月29日 00:31

📌 点击上方蓝字关注公众号：SRETalk，然后点击公众号底部菜单“资料”，可以获取最近两年整理的监控/可观测性相关资料。

Prometheus 告警事件中的 `$value` 表示当前告警触发时的值，但是在告警恢复时，Resolved 事件中的 `$value` 仍然是最新告警时的值，并非是恢复时的值，这是什么原因和原理？是否有办法来解决呢?

不废话，先说原理。

原理

告警规则是配置在 prometheus.yaml 中的，由 Prometheus 负责做规则判定。Prometheus 规则判定的逻辑也很简单，就是周期性的，拿着 promql 去查询数据，如果查到了数据，并且连续多次满足 `for` 指定的时长，就触发告警事件，如果查不到数据，就认为指标是正常健康的状态。比如：

```
cpu_usage_idle < 5
```

如上例，告警规则的 promql 中是带有阈值（`< 5`）的，所以只要查到了数据，就说明当前的值小于 5，只要没查到数据，就说明当前的值大于等于 5，即当前数据是健康的状态。注意，查不到数据的时候，时序库不返回数据，换句话说，数据正常的时候，因为时序库不返回数据，上层就拿不到正常状态时的值，既然拿不到正常状态时的值，也就没法在恢复时展示当前最新的值啦。

实际上，恢复时的事件，是 Alertmanager 根据 resolve_timeout 生成的，而不是 Prometheus 生成的。Alertmanager 生成恢复事件时，会把上次告警的标签和注解带过去，值呢？就是上次告警时的值，Alertmanager 不会再去查询 Prometheus 拿到最新的值。

Alertmanager 可以拿到恢复时的值么?

坦白讲，很难。Alertmanager 需要根据上次告警的标签和注解去查询 Prometheus 拿到上次告警时的值，Alertmanager 不会这么干的，核心是：

- 从职能上，Alertmanager 去查询 Prometheus，就反向依赖了，Alertmanager 是告警的分发中心，不止是接收 Prometheus 推送过来的事件，还会接收其他告警源推送过来的事件，

如果 Alertmanager 要去查询 Prometheus，那就耦合太过严重。

- Prometheus 的告警规则中可以附加标签，和监控指标的标签一起，作为事件的标签集发给 Alertmanager，Alertmanager 需要根据这些标签去查询 Prometheus，拿到原始数据，这在某些场景下是不可行的。一个是要把标签中的附加标签剔除，只留数据标签，Alertmanager 没有办法做到；其次，有些 promql 查询结果压根就没有标签，根本没法查；再次，Alertmanager 需要解析 promql，把阈值的部分拿掉，而有些 promql 压根就不是数字阈值。

如果你想通过修改 Alertmanager 达成所愿，放弃吧。

有没有办法解决?

有。通常的解决办法有两种：

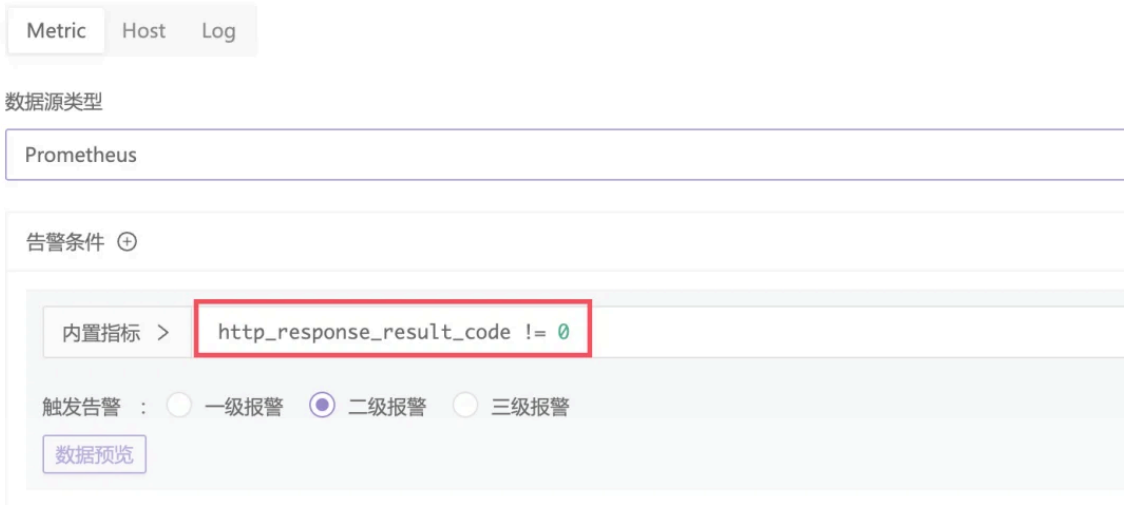
- 告警规则里，顺便配置恢复时的 promql
- 把阈值从 promql 中拿掉，promql 只用于查询原始数据，然后在上层做阈值判定，不管监控指标当前是否健康，都会查到原始数据

下面我以一些监控产品为例，具体来说明。

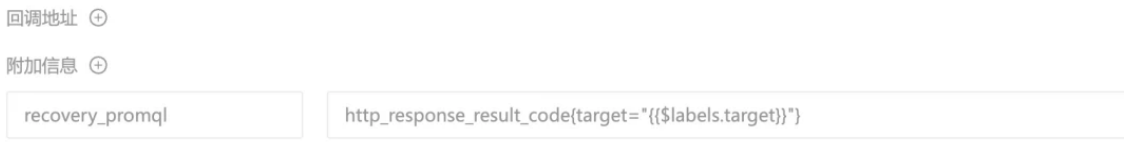
告警规则里配置恢复时的 promql

使用这个方法的产品，以夜莺（Nightingale）监控为例，来讲解具体做法。核心是配置两个地方，一个是在告警规则里配置恢复时的 promql，另一个是在告警模板里配置恢复时的值的渲染。

比如我有一个告警规则用来侦测 HTTP 地址探测失败：



需要在告警规则最下面的自定义字段里，增加 recovery_promql 的配置，如下：



要理解这个工作逻辑，我们先来看看 http_response_result_code 这个指标的数据长什么样子：



从上图可以看出，这个指标包含两个 series，其中 agent_hostname 和 method 字段相同，target 字段可以区分开这俩 series。告警规则 http_response_result_code != 0 如果触发，告警事件中一定会带有 target 标签，所以，如果告警事件恢复的时候，我们用高警时的那个 target 标签去查询，一定就可以准确查到恢复时的值了。所以 recovery_promql 的配置中引用了 target 标签，其值是变量，这个变量就是告警事件中的 target 标签值。

然后，我们在告警模板里，增加一个恢复时的值的渲染，以钉钉模板为例：

```
#### {{if .IsRecovered}}<font color="#008800">💚{{.RuleName}}</font>{{else}}<font color="

---
{{$time_duration := sub now.Unix .FirstTriggerTime }}{{if .IsRecovered}}{{$time_duration
- **告警级别**: {{.Severity}}级
{{- if .RuleNote}}
- **规则备注**: {{.RuleNote}}
{{- end}}
{{- if not .IsRecovered}}
- **当次触发时值**: {{.TriggerValue}}
- **当次触发时间**: {{timeformat .TriggerTime}}
- **告警持续时长**: {{humanizeDurationInterface $time_duration}}
{{- else}}
{{- if .AnnotationsJSON.recovery_value}}
- **恢复时值**: {{formatDecimal .AnnotationsJSON.recovery_value 4}}
{{- end}}
- **恢复时间**: {{timeformat .LastEvalTime}}
- **告警持续时长**: {{humanizeDurationInterface $time_duration}}
{{- end}}
- **告警事件标签**:
```

```
{{- range $key, $val := .TagsMap}}
{{- if ne $key "rulename" }}
  - `{{$key}}`: `{{$val}}`
{{- end}}
{{- end}}
```

这里最为关键的逻辑是判断 `.AnnotationsJSON.recovery_value` 的逻辑：

```
{{- if .AnnotationsJSON.recovery_value}}
- **恢复时值**: {{formatDecimal .AnnotationsJSON.recovery_value 4}}
{{- end}}
```

如果 .AnnotationsJSON 中包含 recovery_value 就展示，展示的时候把 recovery_value 保留 4 位小数。这个 .AnnotationsJSON 是夜莺告警规则中的自定义字段部分，如果告警事件中有恢复时的值，就会在这个字段中体现。

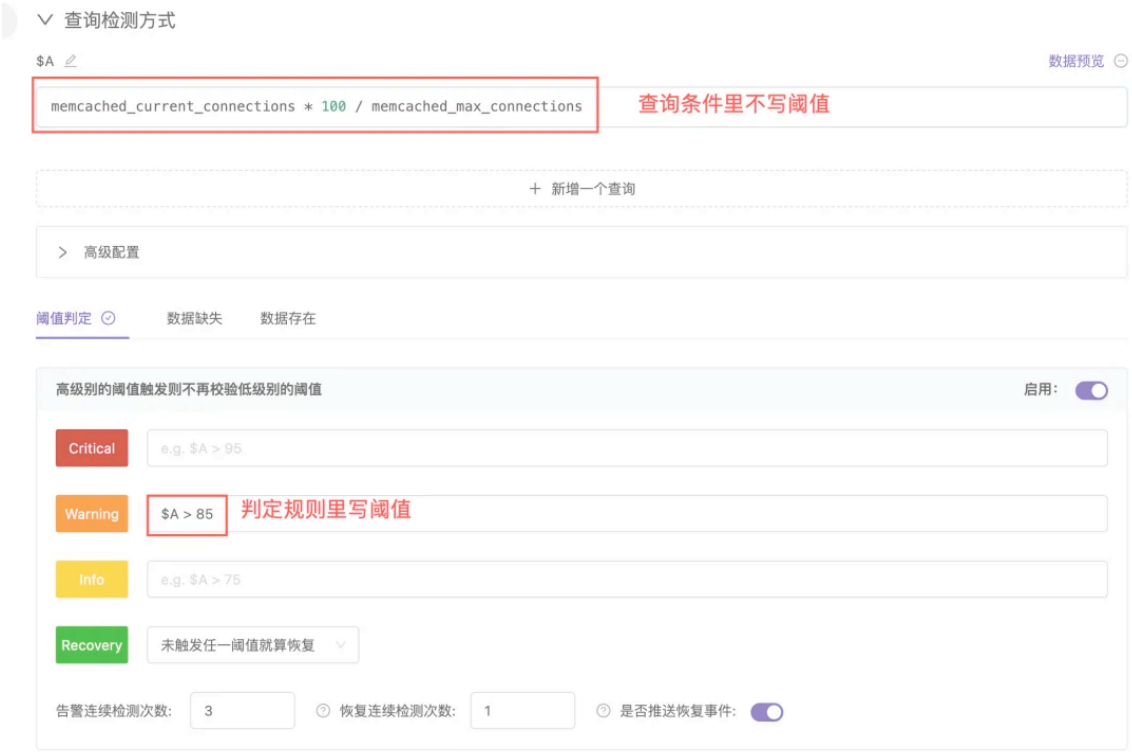
最终效果如下：



把阈值从 promql 中拿掉，promql 只用于查询原始数据

使用这个做法的产品，以 FlashDuty 举例，FlashDuty 不但支持类似夜莺这样的配置 recovery_promql 的方式，还支持 promql 中不带阈值的方式。我们重点讲解 promql 中不带阈值的方式。

以 Memcached 的某个告警规则举例，查询条件里不写阈值，判定规则里写阈值，如下图所示：



这种方式需要先查到当前值，再拿着当前值去做判定，所以不管是告警时还是恢复时，都可以拿到当前值。这种方式非常直观，大部分场景都适用。对于一个查询条件过滤到很多时序的场景，这种方式会查到特别多的数据，对告警引擎也是个压力。需要斟酌。

如果你也想让自己的监控系统支持在恢复时拿到恢复时的值，可以参考上述两种方式。

更多好文：

- 7 张图，彻底讲透 Prometheus 架构原理，收藏备用
- Kubernetes 中必备的 10 个告警处置方法
- 用实际例子讲透 Kubernetes Pod
- SRE 必备知识 - Kafka 探秘之零拷贝技术
- Grafana Loki 架构详解，成本比 ES 低很多
- 如何在 Kubernetes 里部署 JMX Exporter
- 一张图深入分析开源监控夜莺架构
- 监控都没做好，你还要可观测性...


巴辉特
中国大陆
扫一扫上面的二维码图案，加我为朋友。
您好，我是本公众号主理人秦晓辉，开源监控项目 Open-Falcon 和 Nightingale 的发起人，极客时间专栏《运维监控系统实战笔记》作者，目前在创业，为客户提供监控/可观测性解决方案。左侧二维码是我的个人微信，欢迎加我好友一起交流，备注暗号“老铁”，并备注您是谁 ☺